Week 04 Assignment

English Dictionary – Autocomplete Search

Korea University Sejong Campus, Dept. of Computer Science & Software

Assignment Overview

- **Weight:** 1% of total grade
- **Deadline:** Monday before the next theory class, **11:59 PM**
- **Submission:** LMS (source code `.zip` + report `.pdf`)

> **Important:** LMS may experience issues near the deadline. Submit by **Sunday** at the latest to avoid problems. Late submissions due to system errors **will not be excused** – meeting the deadline is the student's responsibility.

Problem

Build an **English dictionary** web app with real-time autocomplete. Compare **linear search** vs **binary search** to demonstrate how divide and conquer improves search performance.

Backend: Python (FastAPI or Flask) **Frontend:** HTML + CSS + JavaScript

Required Features

1. Search Algorithms

Load a word list (10,000+ English words) and implement **two search approaches**:

Approach	Method	Complexity
Linear Search	Scan all words sequentially for prefix match	O(n)
Binary Search	Sorted array + binary search for prefix range	O(log n)

The backend loads the word list on startup and serves search results.

Required Features (cont.)

2. Web Interface

- A **search bar** – as the user types, results update in **real time** (autocomplete)
- Display **both** search results side by side:
  - Linear search results + time taken
  - Binary search results + time taken
- Show the **speedup ratio** (linear time / binary time)

3. API Endpoints

Method	Endpoint	Description
`GET`	`/search/linear?q=prefix`	Linear search autocomplete
`GET`	`/search/binary?q=prefix`	Binary search autocomplete
`GET`	`/`	Serve the frontend page

Deliverables

1. Source Code (`.zip`)

```
week04_assignment/
├── app.py              # Backend
├── words.txt           # Word list (10,000+ words)
├── static/
│   ├── index.html
│   ├── style.css
│   └── app.js
└── requirements.txt
```

2. Report (`.pdf` only, max 2 pages, font size 11pt+)

- **Screenshot** of the autocomplete dictionary in action
- **Performance table** comparing linear vs binary search at word list sizes 1K, 10K, 100K
- **Analysis:** Explain why binary search is faster and connect to the divide and conquer strategy

Grading Criteria

Criteria	Weight
Web app runs and works correctly	20%
Algorithm requirements fully implemented	40%
Report (2 pages max, font size 11+)	40%

Notice: Assignment content may appear on **written exams**. If there is a significant discrepancy between your assignment results and your exam performance, a **grade penalty** may apply.

Submit to LMS before the deadline.